
AI-Enhanced F1 Simulator

Individual Report

Author:
JUNJIAN CAO

Supervisor:
RUZANNA CHITCHYAN

Department of Engineering

UNIVERSITY OF BRISTOL

An individual report submitted to the University of Bristol as part of the requirements of the degree of MASTER OF SCIENCE, describing my personal contribution to the group project AI-Enhanced F1 Simulator.

SEPTEMBER 2026

Word count: approximately 5,600 (body text, Sections 1–6)

Abstract

The given report is a description of the portion of the group project AI-Enhanced F1 Simulator that I undertook. My teammate and I, Songyun Han, worked on the Live Commentary feature. It is one of the four AI features in the project.

My main task was the part of the system that followed the point where the simulator identified the occurrence of a race event. Each chosen event became a prompt to the model. I also supervised the playback of the speech and captions, making sure that in a busy race moment there was no silence or overlapping speech. To ensure that it really did work like this, I made the tests.

Following the brief introduction, my report consists of five major sections. The first one is where I will describe what I have done in this project, the concepts and new strategies behind the design of the features. Secondly, the report explains how I implemented these features. This will include how I determined, during system load, the choice between preempting an event on the queue with a newer event or dropping the newer event. It will also cover how I designed the prompts. Third, I lay down how I would test the Commentary feature and the tools I would use to ensure that it had been tested appropriately and consistently. Fourth, the report highlights how my partner and I organized our work, how we divided the tasks among us and when we planned each stage of our work. Lastly, I critically analyse the work and look forward to the future work.

Author's Declaration

I declare that the work in this report was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in this report are those of the author.

SIGNED: DATE:

Table of Contents

1	Introduction and Context	1
2	Innovation and Creativity	2
3	Technical Development	3
3.1	Analyze the problem	4
3.2	First iteration: silent prefetch	4
3.3	The second iteration is a queued slot with priority replacement	4
3.4	Field-tagged prompt formatting	5
3.5	A similar deficiency: context- budget headroom	6
4	Evaluation Skills	6
5	Project Management Skills	8
5.1	How to fit the personal planning in a cadence of the team	8
5.2	Ordering of personal tasks.	8
5.3	Working through the interfaces of the group	8
6	Critical Self-Evaluation and Future Work	9
6.1	Withdrawing bilingual commentary	9
6.2	The one highest priority per cycle limit	9
6.3	Naming as an interface	10
6.4	Future work	10
A	Code Figures	12

1 Introduction and Context

The name of our group project is AI-Enhanced F1 Simulator. It is a racing simulation with AI capabilities, developed over TORCS ^[1], which is an open-source driving simulator providing real-time data. Similar to other real-time racing simulators, TORCS creates telemetry at a rate that cannot be read by a human during driving. That is because the position of a car, its damage, lap time and its distance to other cars are being updated dozens of times per second. In a real race, there is a broadcast commentator who does the same work. The commentator consumes much more information than a spectator can keep up with. They select the tiny portion that actually counts and formulate a sentence to be communicated to spectators. This enables the spectators to rapidly comprehend what is going on on the track. The Live Commentary module acts in this capacity in this project. Work on it was done with my teammate, Songyun Han.

The feature caters to two sets of users: drivers and spectators. The driver should focus his/her attention on the screen of the simulator. Commentaries in the form of text on screen would divert such focus, so we opted to use a channel that does not compete with visual focus: speech. Voice output allows using a voice line informing a driver of a competitor coming up behind him or of his own position changing to help him know how the race has been altered. They are never required to take their eyes off the track ^[3]. To an observer who is watching another person drive, or watching the AI Driving Bot race by itself, the feature works in the same way as a commentator would have in a live broadcast. It is something that none of the other three features of the project try to achieve. Text combined with speech helps meet the needs of spectators to keep up with information that is constantly changing. Drivers and spectators alike do not desire a system that keeps speaking to them or repeating its message. It is because, as it will be explained in the subsequent paragraphs, much of my work involved not producing text. It involved choosing and timing it.

The Live Commentary is one of three features which require the same set of real-time simulator data. The remaining two are the Race Engineer and the Telemetry Coach. Rather than having each feature open its own connection to TORCS, the project developed a common layer to gather this data. The telemetry is sent over a read-only UDP interface. A single TelemetryService will parse this and put it into a thread-safe TelemetryStore. Each of the three features can access this store without contention due to the same socket. Live Commentary is constructed over this shared store. All the rest of it below it is reserved to Commentary: event detection, prompt construction, and output scheduling. That section of the system was created solely to narrate, not diagnose or advise. The only point at which any of these private elements pass back into shared infrastructure is when it is called as a model call. The actual call passes through the projects shared ModelBroker. My contribution will cover this private layer. I describe how one chosen race event is converted into a bounded prompt, how its output is scheduled such that it does not go silent or speak over its own voice, and what tests I have performed on this pipeline.

The structure shown in Figure 1 is an end-to-end view of the following: the common infrastructure at the top; in the middle, the place where the payload construction of Commentary differs with the way the other two telemetry-fed features construct their prompts; and finally, the private output-scheduling pipeline at the bottom. The pipeline is discussed in detail in Section 3.

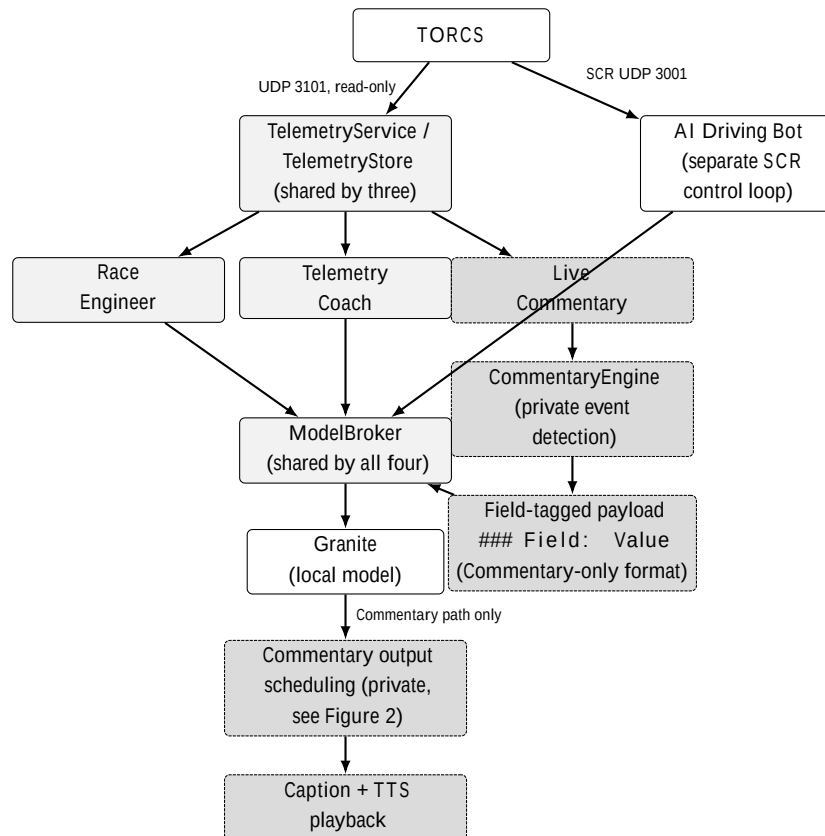


Figura 1: Compartida workflow (cuadros sólidos) contra el trabajo privado del Commentary (cuadros punteados). Telemetry acquisition is a common feature of three features with the model broker being shared by all the four. The TORCS control interface that is used by the AI Driving Bot is SCR (Simulated Car Racing).

The approach adopted by the AI Driving Bot will indeed alter the manner in which the car is driven. Commentary, however, is not the same: its output never enters the control loop. In the worst case scenario, an excessively slow or completely inaccessible model call causes only one race to go unreported. The simulation is not broken. Commentary simply observes the race. It never takes control of the car. This restricted functionality explains why the feature is important. It allows you to interpose a language model between a high-speed simulation and its user. The reaction time of a language model can be expressed in seconds, and the outcome cannot be completely guaranteed. The same limited and passive role might also be applicable outside of racing: in any situation where a real-time system generates information at a rate exceeding what a human can keep up with, a language model may be used as an intermediary of human-readable information without ever having control over the situation. This would include live sports data, industrial monitoring, or a real-time dashboard, etc. Sections 3 and 4 examine this border more carefully. They explain how the system prevents a pending or delayed model response from interfering with the underlying telemetry stream at all. Besides, I developed the tests to ensure that such was the case, in actual faults and faults that I intentionally induced.

2 Innovation and Creativity

On the Commentary feature we divided the labour by stages where he took care of the initial architecture and I took care of the feature development on top of it. Han constructed the module which determines if a telemetry change should be narrated. There are three of them: the sliding-window event detector, the ranking of various events according to their priority and the rules of cooldown and deduplication. The rules

prevent the repeated narration of the same battle or the same off-track episode with each telemetry tick. To put it concisely, it answers to one question only: is the given event worth telling about? Section 4 will validate that it is effective in both the test plan my teammate designed

and my own checks against real driving footage.

However, this design does not answer one other issue: if an event is deemed worthy of being narrated, is it also worth playing at this moment? When this occurs, Commentary requests a line from Granite. Nevertheless, before this first line finishes producing or playing, a more significant event may occur. Initially, in the pipeline, the same logical code answered both questions. That is, the system viewed the two concepts, worth narrating and worth playing right now, as identical decisions. It was all good when the commentary was infrequent. But then it failed precisely when commentary was most needed: in the middle of a multi-car melee or a round with lots of contact and recovery off-track. At such times, there might be multiple events that could be worth narrating within the few seconds it takes to come up with and read out a single line.

I observed this issue during one of the initial demos and discussed it with Han. Looking at the recording together we concluded that the question of whether something was worth narrating and whether it was worth playing right away were not exactly the same question. We explored several methods to address this issue and came up with the conclusion that Commentary should have its own output-scheduling layer: two smaller state machines coupled by a simple interface, not just one large machine. I wrote and implemented this layer myself, as an interrupt/queue playback mode, and was placed completely after the event/interval/hybrid trigger logic of Han. It allowed me to consider and experiment with two questions separately, which were: is it worth narrating, and what becomes of the speech already queued when something else more important comes along. Section 3 elaborates further on this scheduler. The main creative move was to understand that what appeared to be one decision was in reality two distinct questions. It had not been apparent earlier: as mentioned earlier, the common logic did its job until the importance of commentary reached its peak, and failed only in actual racing scenarios. When I developed the second state machine, only its effects were present - playback would go silent or poorly timed when multiple events occurred in quick succession.

A less extensive manifestation of the same concept is not to adopt anything merely because it is already there. It pertains to the process through which a certain event makes its way to the model. In our initial construction of this component, Han and I merely composed the event payload in JSON and sent it to the model in its unmodified form. The format itself was not given much thought. Afterwards, we talked about it and understood that such a choice might be an issue. It influenced the token cost of any request as well as the quality of the output produced by the model. The brackets, quotes, and indentation of JSON have nothing to say about the race. Yet the model needs to read them and at times reiterate them in its output. After this became a real issue rather than a guess, I looked into how others had done so the same work: transforming structured information into a prompt. I came across a published system that employed the same convention in a very similar task. This is elaborated further in section 3. The same concept was applied here- text/plain with field names instead of JSON. E.g., `### Speed change: +38 km/h` as opposed to `"speed_delta": 38`. This format does not contain all fields that are irrelevant to the particular event type but sends the same full payload every time.

3 Technical Development

This part provides greater information on the components of the Commentary feature that I coded myself, particularly the output scheduler and the prompt-building surrounding it, in comparison with the group report. It talks about, first, my reasons behind designing the scheduler and how its design evolved through two iterations; second, the field-tagged prompt structure that I used to create every request; and lastly, a context-budget issue that I discovered in testing and how I solved it.

3.1 Diagnosing the problem

The next subsection defines the position of scheduler in the pipeline besides what it should do. The upstream process of event selection determines if a telemetry change should be reported and was implemented by Han (described in section 2). The downstream process requires that the scheduler should maintain two outputs synchronized, i.e., the caption that is displayed on the shared dashboard and the voice that is spoken through the text-to-speech (TTS) system. Its task is as follows: after receiving a selected event the scheduler needs to ensure that both caption text and spoken commentary are presented simultaneously.

On an early, single-slot version, the scheduler was able to retain only a single pending request at any given time. A new event either would wait until the existing request completed, or, in interrupt mode, would cancel the request and began building a new one afresh. It was this second behavior that resulted in the issue. Each time the scheduler killed one request and created another, the dashboard did not display a caption and the TTS service did not play any sound, until Granite had finished creating the first token on the new line, which might take some time. Two or three events occurring in quick succession are precisely the time when commentary is most valuable. The scheduler did not output quicker commentary in this case. Rather, there was an interval without any caption or audio, then a line containing a description of only the final event.

3.2 First iteration: silent prefetch

First of all, my initial fix focused on the silence issue, as it occurred every time when the scheduler killed the existing request and began creating a new one, which caused the dashboard to have no caption and the TTS service to have no audio until the next line was prepared. Rather than cancelling the in-flight request altogether, another, higher-priority event caused another, "silent" generation request to be made to the ModelBroker immediately. The existing line was permitted to end rather than being interrupted in the middle of a sentence. In this case, "silent" meant that the request was performed in the background, and its tokens accumulated there. No output was delivered to the WebSocket or to the TTS service until it was that request. It implied that the dashboard and TTS service would never be empty during the creation of a new line, but introduced a new challenge. With only one slot available, a third event coming in while the silent request was still being generated could not go anywhere other than to cancel it. It had returned the original problem with one level of removal, which is now bursts of three events rather than two. Figure 3 illustrates how the function of this silent request the prefetch evolved into: `_start_prefetch()` in `middleware/runtime.py`. It cancels any previous prefetch that is still running, then starts a new one whichever event is currently queued.

3.3 Second iteration: a queued slot with priority replacement

The last version cannot cancel the line already playing in any form. Two things are tracked by module-level state in `middleware/runtime.py` separately: `_commentary_task`, which is the coroutine that is currently generating, or has already started playing, and `_pending_decision`, which is at most one event waiting to be used when the current line ends. When a new event comes in during the busy time, `_queue_event()` (Figure 4) takes place. In case its priority is less than the one that is currently present in `_pending_decision`, it is immediately thrown away. Otherwise, it substitutes the vacant spot, and `_start_prefetch()` is invoked again to ensure that the silent generation remains synchronized with what is currently pending. It was not a decision solely made by me. After an experimental version, Han and I agreed on this design based on

discussions, as advised by our supervisor to use a queue (Section 4). We observed that a queue that received each collected event lagged further and further behind a live race at busy times. The conclusion we reached was that it would be preferable to drop a lower-priority event rather than have the scheduler eventually describe something that had previously occurred several seconds ago.

This type of issue is also known by a particular term in the systems literature: load shedding. This implies making a choice about which things to throw away when a system can no longer handle all that enters it. Once Han and I agreed upon abandoning, not queuing, I searched how this problem has already been researched elsewhere. I came across work by Tatbul et al. on load shedding in a data stream manager [7]. They confront a comparable dilemma but on a much bigger scale. If an overloaded stream processor cannot keep track of all tuples that arrive at it, it has to determine which of them to drop. Their solution is to drop it on whichever of the output streams has the smallest effect on quality, rather than randomly discarding it or distributing the damage evenly across all streams. My own `_queue_event()` logic is, essentially, a miniature version of a drop decision: it does not treat each of the events in the queue as something of equal value to keep, but rather maintains the significant one, and removes the less significant one. What makes it a principled drop, as opposed to an arbitrary one, are the event priorities: the same rationale behind the decision by Tatbul et al. to shed load where quality is lowest.

Upon receiving a message from the front-end indicating that playback has ended, `_advance_queue()` (Figure 5) releases the pending slot. In case the system is not in queue mode, or there is nothing pending, it just resets the busy flag and becomes idle. In other cases, it assumes the pending decision and its prefetch task. It waits until such prefetch is done, unless it has already been done, before sending the result as the next caption and audio clip via the `_broadcast_silent_result()` method.

The final pipeline described in the figure 2 starts with a chosen event and ends with the caption and audio which ultimately reach the end-user.

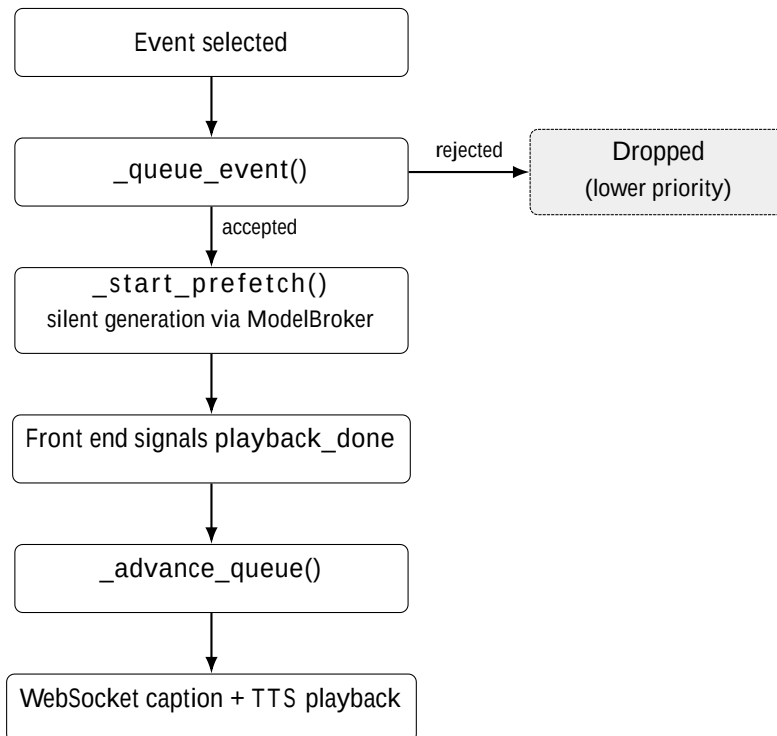


Figure 2: Final output-scheduling pipeline starting with a chosen event and ending with the caption and audio being given to the user. The dashed box indicates the one point of discarding an event instead of carrying it on.

3.4 Field-tagged prompt formatting

The field-tagged rendering mentioned in Section 2, which will write every item of event data as `### Field: Value` instead of as JSON, is a concept that Han and I came up with after testing and thinking about it. Our investigation involved the study of how big language models digest input and how they ingest tokens, and

we discovered that originally

The cost of the JSON format could have been both tokens and the generation quality. I thereafter searched how others had addressed a comparable issue and discovered the paper by Cook and Karakus, on fine-tuning a language model so as to produce football commentary using structured match-event data ^[6]. The system also processes every data that is given to the model in its input with a prefix of `##`, and it is done so that the model can ensure that it can determine the end of a piece of event data and the start of another. It is what made me choose the same strategy in this case, instead of coming up with anything new.

In this project, this rendering technique is applied in another way than in the mentioned literature. Their prefix denotes training examples which are baked into fine-tuned weights of a model. Mine denotes a payload that is built new with every request, compared to a model that has no fine-tuning whatsoever to Commentary. Since this project was unable to fine-tune the Granite model, the same formatting concept had to play a different role in this setting. Rather than instructing the model a pattern in training, it has to inform the model that pattern at inference time, regardless of the current knowledge of the model about reading structured text.

3.5 A related defect: context-budget headroom

The context-budget error was not identified as I considered the design. It was identified in one of my manual testing iterations (Section 4), where a request that was expected to succeed failed. The ContextManager constructs the prompt of each request out of the persona, the event payload, and a small portion of past commentary history. It uses a fast heuristic to estimate how much the text will cost in terms of tokens: approximately one token for every four ASCII characters and one token per every two CJK (Chinese, Japanese, and Korean) characters. It does not utilize the actual tokenizer of the model, since the operation of a real tokenizer on any frame would be excessively slow with a locally hosted model. This heuristic is at best an approximation. In a lengthy testing window, it would occasionally under-estimate a big history slice by exactly the amount needed to take the prompt beyond the actual limit of the model. In such cases, the request was refused outright, rather than truncated further.

The initial iteration of the budget did not allow any error of this nature. The historical budget token was calculated precisely as `max_context_tokens-max_response_tokens`, and in case of estimation, there were no remaining tokens. It is an obvious mistake to have made on the first attempt, as it appeared to be a modelling issue, not a budgeting one. My solution was minor; to cut the history budget by a fixed safety margin (`safety_margin_tokens`). That is, the heuristic can be slightly off, but the request will not fail. To ensure that the fix really worked, I intentionally repeated the failure in a subsequent test round, rather than simply assuming that it had.

Looking back, these two pieces of labor, namely the queued-slot scheduler and context-budget fix, together represent the two types of issues that have been recurring with me on the Commentary function. One of them was architectural, due to two jobs utilizing the same mechanism. The other one was an error in the way in which the code surrounding a heuristic treated its error. Neither of these issues is discussed in detail in the higher-level description of the pipeline in the group report. It is the reason why I decided to explain them correctly in this post.

4 Evaluation Skills

Commentary test plan, which my teammate, Han, did was the one that defined the categories of events that needed to be covered and pass or fail criteria based on the accuracy of detection and priority behavior. It was my task to execute this plan against the completed implementation, discover the points of failure,

and correct the code. In order to achieve this, I had written the test code by myself: most of them were automated tests, with each test feeding a manually constructed telemetry frame to the detector and then verifying

that it restored the correct type of event. It implied that I was able to repeat the checks within a few seconds after a code alteration, rather than running a whole session each time. I have also written a monitoring script, logging all events that the system reported, each with a timestamp, and another script matching these logs with what I manually labeled the events to calculate detection accuracy. Besides, I verified personally whether what the system claimed to detect corresponded to the reality of what occurred during the drive, rather than relying on the log. It was important because a language model may generate a sentence which seems plausible, even though it does not reflect the reality depicted in the telemetry data ^[2].

This feature was not assessed based on my test plan alone. One design decision was made as a result of formative feedback by our supervisor. At the close of the project, when we were getting ready to conduct our scheduled user evaluation, our supervisor requested us to demonstrate the feature at a group meeting. She was disturbed by the immediate-interruption delivery. She was not satisfied with the manner in which a new event would interrupt an existing line and she recommended dealing with small events bursts through a queue. This forced me to revise the mechanism, which was no longer the instantaneous interruption but to queue new lines instead of terminating the current one. However, when I began constructing it I discovered that a queue that accepted all events lagged behind the live race in times of high activity. Therefore, I created a one-slot system with priority-based replacement, not queueing all events; the decision to drop less important events, not queue them, was a joint decision between Han and me, as explained in Section 3. This is formative feedback to shape a design iteration, not the outcome of our intended user evaluation.

Four times during the development process I went over the entire plan by hand, as opposed to once at the end of the development. On all of the first three rounds, I identified an actual issue. This was a duplicate comment line, which came about due to the check of deduplication being run after the broadcast of the WebSocket, rather than prior to it. Another was an ineptly adjusted pace surge threshold that would go off with normal acceleration. The third one was the context-budget rejection which I wrote about in Section 3. Every single time, I would go back to the specific code which generated the issue, resolve it and replay that segment of the plan then proceed. It was only in the fourth round that all of the elements in the plan passed smoothly. At that point, I felt the feature was acceptable, based on the standards proposed by Han.

The most intensive effort I made regarding this step was to ensure that a positive result was correct, not merely self-consistent ^[4]. It is possible that a detection script can register a series of events that are apparently correct, although there is actually something wrong with them. It could record a contact event, but fail to detect an off-track excursion occurring simultaneously. None of it is disclosed in the log. In order to verify this, I recorded the precise times of start and stop of every test drive and video-screened the entire session. Then, I checked the CSV file of the monitoring script entry by entry based on the timestamps to find out exactly which of the logged events corresponded to which moment of the recording. It is thus how I verified, rather than simply presumed, that the events reported by the script were consistent with what really took place on the screen. Additionally, it allowed me to identify a genuine constraint that the log could never have revealed. Both of these events were captured in the recording when an off-track excursion occurred during the same cycle as a contact event. But since the detector can only store one match per cycle and the two events had the same priority, one of them became silently discarded: the contact event was recorded, but the off-track excursion event was not.

Considering all these things collectively, such as passing all the criteria of Hans test plan, being able to be validated with real footage as opposed to a self-consistent log, and surviving with real faults and faults I have caused deliberately, this makes me confident that the feature is of high quality. In Section 6 below I talk about the priority-collision limitation and what I would change concerning it.

5 Project Management Skills

Project management is already discussed in group report on team level. The discussion in this section explains how I had individually planned, managed, and contributed to the development of the Commentary feature with Han, and what form this two person collaborative approach to work took and how it influenced my own approach.

5.1 Fitting personal planning into the team's cadence

We have developed using the SCRUM methodology ^[5]. The sprint was organized in each week. Our academic supervisor, Ruzanna Chitchyan, was informed about the work done during the last week by each team member on Mondays morning, where he or she provided feedback about the future direction of the project. Immediately after this, the entire team gathered and held an internal planning session on that Monday evening. This allowed task allocation during the next week to respond to the most recent remarks made by the supervisor, rather than being several steps behind him or her. It was during this meeting on Monday afternoon when Han and I agreed on a week by week basis who will do what work on Commentary. We recorded the outcomes on a shared Jira board, and not in a personal list that only the two of us would use. We also were sending a written progress report by email to our academic and IBM supervisors at the end of each week. I needed to plan my weekly work in advance, because I should have been able to report on what was achieved at the group meeting. It allowed me to obtain the feedback of our supervisor on time and apply it to enhance my design.

5.2 Personal task sequencing

Within that weekly framework, I would build my own work in layers, rather than attempt to create the entire commentary pipeline at once. Every layer needed to run end to end before I could add more complexity above it. The first thing I did was construct a tiny middleware service and a ContextManager that could accept live telemetry and generate one spoken line, a simple caption and a voice front end. The team had something functional in less than two weeks, even though each part of it was still unpolished. After this I went back and enhanced what counted in regards to quality, namely the prompt design and the context budget, and finally the interrupt/queue scheduling layer as described in section 3 when the previous, simpler mechanism could no longer cope with the pace at which events took place in a real race. To isolate these changes between each other, I employed a Git branch within our shared Commentary feature branch (voice2.0, voice6.0, commentary4.0, etc.). This allowed experimentation with the prompts or playback logic without disrupting a version that Han, or anyone else on the team, was already using. It was also aligned with the rule of the team: new changes are only merged into the branch that is visible to the rest of the team after being tested on over one machine.

5.3 Working within the group's interfaces

The commentary relied on infrastructure which belonged to other people: the event-detection logic of Han, the shared TelemetryService, and the shared ModelBroker of every feature. This is why much of management of my own work involved monitoring changes to these upstream components instead of relying on specifications that were determined at the beginning of the project.

One instance that arose when I was re-engineering ContextManager was that I intended to replace the `commentator_persona` field with another more understandable identifier. I did not do so. The precise name was

already a component of the API that the dashboard configuration panel had been implemented by Shangqing Li in Race Engineer. To change it would have disrupted the functioning of the system that I did not own and therefore would be unable to test easily. Then I retained the name and enhanced the code surrounding

However, I am not going to do it. It is a minor case in point but, I believe, a good one: any other teammates code relies upon an interface. When I can come up with a better solution on my own, I should also talk it over with my teammate before implementing it.

This occurred also at the output end. My caption and voice front end and subsequently a different Electron overlay were swapped out over time. This was after the team settled on a single shared HTML dashboard that would apply to all four features. Having done so, I transferred the playback logic to it and removed the old and separate Commentary page, instead of having two pages. I could see when I had put together something on my own and it was replaced by a larger scale of team effort. It was also a decision to migrate cleanly, rather than to defend my own component.

6 Critical Self-Evaluation and Future Work

Reflecting on this project, there are three aspects that are worth considering. The first is the removal of bilingual commentary, a joint decision between me and Han after we discovered an issue and tested it ourselves. The second is the highest-priority-per-cycle limitation, which became apparent as soon as we tested it (Section 4) and can be traced to a design decision I took without any doubts. The third is a more minor, continuous naming issue with regard to the pipeline that better cooperation at the beginning would have eliminated.

6.1 Withdrawing bilingual commentary

Initially, Commentary provided two language choices to speak its narration in: English and Chinese. However, the Chinese half, including the text and audio, was of a poor quality. The Chinese commentary could be erroneous in changing the meaning or misusing the words. Meanwhile, the Chinese TTS voice was unnatural, and much worse than the English voice. Having discussed this with Han, we concluded that none of us could be satisfied with the design. The team also saw it, and the reviews were negative as well. Therefore, we have abandoned it and just remained in English. In case the development of bilingual, or multilingual, commentary will continue in the future, I would start by establishing a formal test of translation quality. Besides, I would be ready to test new voice packs, in Chinese or another language, when better ones are released.

6.2 The single-highest-priority-per-cycle limitation

Chapter 4 also explained how matching test recordings to the detection log revealed a situation where an excursion off track occurred in the same cycle as a contact event. The detector stores only one match per cycle and the three events, namely contact, off_track and position_change, have the same priority, hence the comparison cannot distinguish between them. Survival is not based on the priority, but on an undocumented, order-based tie-break in the detection code that always retains contact and eliminates off_track. Off-track recall is reduced exactly when off-track and contact occur simultaneously. Our own test code has even noted this: a comment says that it actually pins the actual undocumented tie-break policy instead of presuming one. I considered it as a priority rule and did not verify whether the priorities were really equal or whether the actual tie-break was undocumented. It was only with testing that I realized some of its price tags. The event log generated by this design can not be regarded as a full record in case two events occur simultaneously which is quite common during precisely such aggressive driving that makes commentary useful. Were I to recreate this once more, I would have split early what logs and what

speaks loud much sooner. I would have recorded each detected event

Even when only one of them is narrated in a given cycle within a structured log, so that testing is not based on the same single-slot decision as scheduling. Future-work section of the group report points to the same direction of Commentary, and this is where I would start working next if I had extra time.

6.3 Naming as an interface

Another issue that always arose was naming. Event, interval and hybrid are the trigger modes of Han. The names of the output-scheduling modes I created are interrupt and queue. Each of the two groups of names is meaningful by itself. However, the person who reads the code without this report cannot tell that these are two different layers of the same pipeline, but two alternative versions of the same thing. On several occasions throughout integration, a colleague questioned me about which of the two configurations applied to which aspect of the behavior. With hindsight, it would have been virtually free to reach an agreement on a common naming convention between me and Han before the choice of either group of names. This would have clarified the two-layer structure in the code itself, rather than in a report done after the fact. As an illustration, we could have applied the same prefix to the trigger settings and another one to the scheduling settings.

6.4 Future work

Other than what I have stated, I can identify two obvious next moves that can be undertaken by Commentary. The first one is a visualization window, which has been directly incorporated into the simulator so as to show the caption within the window rather than merely on the shared dashboard. The second builds upon this idea: an automated test script. By using screen image recognition on this embedded window, the script could automatically compare the events it had recorded with what had actually occurred in place of the manual check based on timestamps and recordings described in Section 4. In case further feature modules are introduced to the project in the future, such automated check will allow making the testing much more efficient.

References

- [1] B. Wymann, E. Espié, C. Guionneau, C. Dimitrakakis, R. Coulom, and A. Sumner, “TORCS, The Open Racing Car Simulator, v1.3.5,” 2013.
- [2] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023, doi: 10.1145/3571730.
- [3] M. R. Endsley, “Toward a theory of situation awareness in dynamic systems,” *Human Factors*, vol. 37, no. 1, pp. 32–64, 1995, doi: 10.1518/001872095779049543.
- [4] J. S. Santillana, “Precision is not faithfulness: Coverage-aware evaluation of grounded generation with a complete oracle,” *arXiv preprint arXiv:2606.09376*, 2026.
- [5] A. Srivastava, S. Bhardwaj, and S. Saraswat, “SCRUM model for agile methodology,” in *2017 International Conference on Computing, Communication and Automation (ICCCA)*. IEEE, 2017, pp. 864–869.
- [6] A. Cook and O. Karakuş, “LLM-Commentator: Novel fine-tuning strategies of large language models for automatic commentary generation using football event data,” *Knowledge-Based Systems*, vol. 300, art. 112219, 2024, doi: 10.1016/j.knosys.2024.112219.
- [7] N. Tatbul, U. Çetintemel, S. Zdonik, M. Cherniack, and M. Stonebraker, “Load shedding in a data stream manager,” in *Proceedings 2003 VLDB Conference*, Morgan Kaufmann, 2003, pp. 309–320.

A Code Figures

```
async def _start_prefetch(decision: CommentaryDecision) -> None:
    ...
    global _prefetch_task
    if _prefetch_task and not _prefetch_task.done():
        _prefetch_task.cancel()
    t, r = telemetry_store.latest()
    request_id = str(uuid.uuid4())

    async def _run():
        return await generate_commentary(
            t, r, event_payload=decision.payload, history_mode="summary",
            request_id=request_id, silent=True,
        )

    _prefetch_task = asyncio.create_task(_run())
```

Figure 3: `_start_prefetch()`, `middleware/runtime.py`: the current in-flight prefetch is cancelled and a new silent generation (`silent=True`) is started on the currently queued event.

```
async def _queue_event(decision: CommentaryDecision) -> None:
    ...
    global _pending_decision
    new_priority = decision.event.get("priority", 0)
    if _pending_decision is not None and new_priority < _pending_decision.event.get("priority", 0):
        return
    _pending_decision = decision
    await _start_prefetch(decision)
```

Figure 4: `_queue_event()`, `middleware/runtime.py`: an event of lower priority is discarded; anything else takes up the vacant slot and restarts the prefetching process.

```
async def _advance_queue() -> None:
    ...
    global _pending_decision, _prefetch_task, _commentary_task, _commentary_priority, _playback_busy

    if commentary_engine.config.interrupt_mode != "queue" or _pending_decision is None:
        _playback_busy = False
        return

    decision = _pending_decision
    _pending_decision = None
    prefetch = _prefetch_task
    _prefetch_task = None
    _commentary_priority = decision.event.get("priority", 0)

    async def _release():
        try:
            result = await prefetch if prefetch is not None else None
        except asyncio.CancelledError:
            result = None
        except Exception as e:
            log.warning(f"Queued commentary prefetch failed: {e}")
            result = None
        if result is not None:
            await _broadcast_silent_result(result, event=decision.event, payload=decision.payload)

    _commentary_task = asyncio.create_task(_release())
```

Figure 5: `_advance_queue()`, `middleware/runtime.py`: it will release the pending slot once the front end signals `playback_done`, depending on whether the prefetch task has not been completed yet or it is waiting on the prefetch task to finish.

